GCard: A Practical Cardinality Estimator for Graph Queries

Paper ID: XXXX

ABSTRACT

This paper presents GCard, a practical cardinality estimation framework for graph queries.

1 INTRODUCTION

Related work. We categorize the related work as follows.

2 PRELIMINARIES

We use the following notation: Σ is a set of labels, Γ is a set of property names, and $\mathcal{D}$ is a (possibly infinite) domain of property values.

Property Graphs. A property graph is a tuple $G = (V, E, \lambda, \rho)$, where V is a finite vertex set, $E \subseteq V \times V$ is a set of *undirected* edges, $\lambda : V \cup E \to \Sigma$ assigns labels to vertices and edges, and $\rho : (V \cup E) \times \Gamma \to \mathcal{D}$ is a partial property function. We use x.prop for the value of property prop on a vertex or edge x.

Subgraph Matching Queries. A subgraph matching query is represented by a pattern graph $Q_S = (V_Q, E_Q, \lambda_Q)$, where V_Q and E_Q are the query vertex and edge sets, and λ_Q is the query labeling function. A *subgraph match* of Q_S in G is an injective mapping $M : V_Q \to V$ such that $\lambda_Q(u) = \lambda(M(u))$ for all $u \in V_Q$, and for each $e = (u, v) \in E_Q$, there exists $e' = (M(u), M(v)) \in E$ with $\lambda_Q(e) = \lambda(e')$. We denote the set of all such matches by $Q_S(G)$. A *path query* is a special subgraph query whose pattern graph is a labeled path. We denote a path query by P and a path query with k edges is written as $P = u_0 e_1 u_1 \cdots e_k u_k$, where e_i connects u_{i-1} and u_i for $i = 1, \ldots, k$. We call u_0 and u_k the endpoints of P.

Graph Queries. A graph query augments a subgraph matching query with constraints on properties of query vertices and edges. Formally, a graph query is a pair $Q = (Q_S, \psi)$, where $Q_S = (V_Q, E_Q, \lambda_Q)$ is a subgraph matching query and ψ is a Boolean formula over literals on query vertices and edges. A *vertex literal* is an atomic condition on a query vertex $u \in V_Q$ (*e.g.,* u.age > 30), and an *edge literal* is an atomic condition on a query edge $e \in E_Q$ (*e.g.,* e.timestamp < 2025). We call both predicate literals. A match of Q in G is any $M \in Q_S(G)$ with $M \models \psi$. For a predicate literal ℓ, $M \models \ell$ iff ℓ evaluates to true under M; for vertex literal on u, evaluate on $M(u)$; for edge literal on $e = (u, v)$, evaluate on $(M(u), M(v))$; $\neg, \wedge, \vee$ are interpreted in standard semantics. Hence, $Q(G) = \{M \in Q_S(G) \mid M \models \psi\}$.

Cardinality Estimation. Given a graph G and a graph query Q, the *cardinality estimation problem* aims to compute an estimate $\hat{C}$ of $|Q(G)|$ without explicitly computeing $Q(G)$.

Example 1: ⇒: Add a running example graph and query to illustrate the definitions and notations. @Xingzi □

3 PATH DEGREE SUMMARY

In this section, we present path degree summary, a core summary data structure of our cardinality estimation framework.

We start with path degrees, which capture the exact distribution of path counts for each vertex. Unless stated otherwise, all summaries are defined *w.r.t.* a fixed data graph $G = (V, E, \lambda, \rho)$.

Path Degrees. Let s and t be the endpoints of a path query P. For each vertex $v \in V$, define $d_s^P(v) = \big|\{M \in P(G) \mid M(s) = v\}\big|$ and $d_t^P(v) = \big|\{M \in P(G) \mid M(t) = v\}\big|$, as the *path degrees* of v *w.r.t.* P, respectively. The *path degrees* of P are the pair (D_s^P, D_t^P), where

$$D_s^P = \{(v, d_s^P(v)) \mid d_s^P(v) > 0\}, \quad D_t^P = \{(v, d_t^P(v)) \mid d_t^P(v) > 0\}.$$

For a degree value $d > 0$, we write $\text{rank}(d) = k$ if $2^{k-1} < d \le 2^k$.

Path Degree Summary. Directly using path degrees for cardinality estimation is computationally and space expensive. We therefore introduce PathDS, a compact summary of path degrees that supports efficient estimation with low space overhead.

Definition 3.1: Let P be a path query with endpoints s and t. The *path degree summary* (PathDS) of P is a pair $(\mathcal{S}_s^P, \mathcal{S}_t^P)$, where

- $\mathcal{S}_s^P = \big\{(k, c) \mid c = |\{(v, d) \in D_s^P \mid \text{rank}(d){=}k\}|,\ c > 0\big\}$,
- $\mathcal{S}_t^P = \big\{(k, c) \mid c = |\{(v, d) \in D_t^P \mid \text{rank}(d){=}k\}|,\ c > 0\big\}$. □

The PathDS summary $(\mathcal{S}_s^P, \mathcal{S}_t^P)$ can be constructed from path degrees (D_s^P, D_t^P) with a single linear scan. Conceptually, $(\mathcal{S}_s^P, \mathcal{S}_t^P)$ is a rank-grouped histogram of (D_s^P, D_t^P) that groups path degrees by rank and records the number of vertices in each rank. This design preserves the degree distribution at rank granularity while substantially reducing space overhead.

Example 2: ⇒: illustrate the definitions with a running example □

PathDS admits a 2-approximation guarantee: replacing each degree in rank k by 2^k yields an estimate in $[|P(G)|, 2|P(G)|]$.

Proposition 1: *Let P be a path query with two endpoints s and t. Then*

- $|P(G)| \le \sum\{2^k \cdot c \mid (k, c) \in \mathcal{S}_s^P\} \le 2|P(G)|$, *and*
- $|P(G)| \le \sum\{2^k \cdot c \mid (k, c) \in \mathcal{S}_t^P\} \le 2|P(G)|$. □

Remark. Degree statistics have been widely used in cardinality estimation for relational and graph queries, *e.g.,* [1–4]. The most related methods are SafeBound [2] and PathCE [4]. We highlight their key differences from PathDS as follows.

(a) SafeBound is designed for relational DBMS and uses ordered degree sequences on join attributes, which are essentially edge-level statistics in graph terms. PathDS differs from SafeBound in two aspects: (i) it focuses on path-level degree statistics in contrast to edge-level statistics, (ii) it summarizes degree distributions into rank buckets, which are easier to compute and maintain than the ordered degree sequences in SafeBound (see also Exp. **X**).

(b) Both PathCE and PathDS use path-level statistics, but they sum-

marize them differently: (i) PathCE partitions vertices into summary groups and stores maximal-degree statistics for each group pair; (ii) PathDS groups path degrees by rank and stores per-rank counts, preserving rank-level distributions with lower space overhead.

4 THE GCARD FRAMEWORK

PathDS: An Update-Friendly Summary.

A Predicate Decoupling Design.

A Cycle-Aware Estimation Algorithm.

5 HANDLING PREDICATES

6 PATHDS BUILD AND MAINTENANCE

In this section, we present a parallelly scalable construction algorithm for PathDS and an efficient incremental update strategy.

6.1 Parallel PathDS Construction

We first present PDSBuild, a parallel algorithm that constructs PathDS summaries for a given graph and a set of path queries .

In a nutshell, PDSBuild computes path degrees in bottom-up manner and updates the corresponding PathDS summaries during the process. The algorithm is parallelly scalable in the sense that it guarantees reduced running time with more processing cores, enabling efficient PathDS construction on large graphs.

Bottom-Up Computation of Path Degrees. Given a path query

Algorithm 1: PDSBuild: Parallel PathDS Construction

Input: Graph $G = (V, E, L)$, a set of path queries $\mathbb{P}$
Output: PathDS $(\mathcal{S}_s^P, \mathcal{S}_t^P)$ for all $P \in \mathbb{P}$

$K \leftarrow \mathsf{MaxLen}(\mathbb{P})$; $\{\mathbb{P}_1, \ldots, \mathbb{P}_K\} \leftarrow \mathsf{GetsSubPaths}(\mathbb{P}, K)$;
Initialize $d_s^P(v), d_t^P(v)$ for all $v \in V$ and $P \in \mathbb{P}$;
for each $i = 1, \ldots, K$ **do**
  for each $P \in \mathbb{P}_i$ **do**
    $(s, s', P_1) \leftarrow \mathsf{HSplit}(P)$; $(P_2, t', t) \leftarrow \mathsf{TSplit}(P)$;
    for each $v \in V$ **do in parallel**
      Compute $d_s^P(v)$ and $d_t^P(v)$ via Eqs. (1) & (2);
      if $P \in \mathbb{P}$ **then** $\mathsf{Update}(\mathcal{S}_s^P, \mathcal{S}_t^P, d_s^P(v), d_t^P(v))$;
return $\{(\mathcal{S}_s^P, \mathcal{S}_t^P)\}_{P \in \mathbb{P}}$;

P, we need the path degrees (D_s^P, D_t^P) to construct the corresponding PathDS $(\mathcal{S}_s^P, \mathcal{S}_t^P)$. A key observation is that (D_s^P, D_t^P) can be derived from the path degrees of immediate sub-paths, which yields a bottom-up dynamic construction from shorter paths to longer ones. Let $P = u_0 e_1 u_1 \cdots e_k u_k$, and define $\mathsf{HSplit}(P) = (s, s', P_1)$ and $\mathsf{TSplit}(P) = (P_2, t', t)$, where $s = u_0$, $t = u_k$, $s' = u_1$, $t' = u_{k-1}$, $P_1 = u_1 e_2 \cdots e_k u_k$, and $P_2 = u_0 e_1 \cdots e_{k-1} u_{k-1}$. Then, for each vertex $v \in V$, its path degrees *w.r.t.* P can be computed as follows:

$$d_s^P(v) = \sum \left\{ d_{s'}^{P_1}(u) \mid u \in \mathsf{Nbr}(v),\ \mathsf{Match}(v, u, s, s') \right\}, \quad (1)$$

$$d_t^P(v) = \sum \left\{ d_{t'}^{P_2}(u) \mid u \in \mathsf{Nbr}(v),\ \mathsf{Match}(u, v, t', t) \right\}. \quad (2)$$

Here, $\mathsf{Nbr}(v)$ denotes the set of neighbors of v in G, and

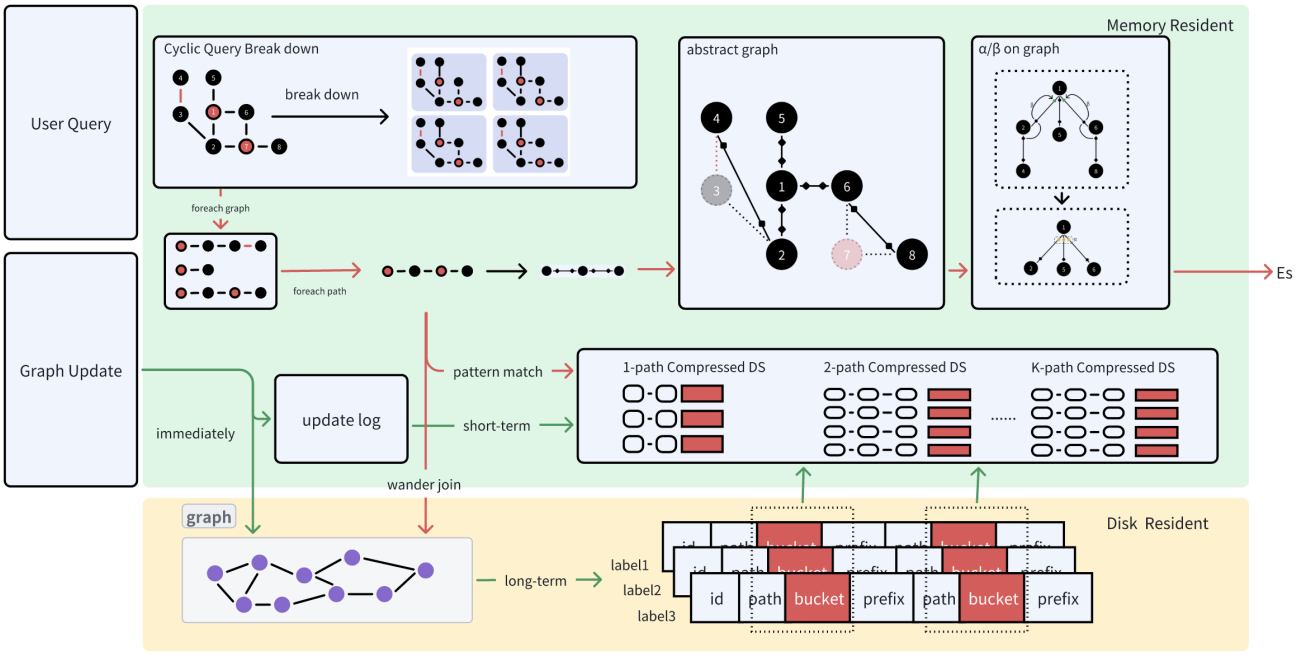

Figure 1: Overview of GCard.

Match(v, u, s, s') (resp. Match(u, v, t', t)) checks whether the data edge is compatible with the corresponding query edge in P. Equations (1) and (2) therefore reduce the computation of (D_s^P, D_t^P) to previously computed degrees on P_1 and P_2.

Parallel Processing. The path degree computation admits two-level parallelism. We can process query groups by increasing path length, execute queries within each group in parallel, and parallelize vertex-wise path degrees for each query since they are independent given shorter-path results.

Algorithm PDSBuild. Putting these together, Algorithm 1 outlines the PathDS construction procedure. Given a graph G and a set of short path queries $\mathbb{P}$, PDSBuild constructs the corresponding PathDS summaries $\{(\mathcal{S}_s^P, \mathcal{S}_t^P)\}_{P\in\mathbb{P}}$ for all $P \in \mathbb{P}$ in parallel.

(1) PDSBuild first generates all sub-paths of queries in $\mathbb{P}$ and groups them by length into $\mathbb{P}_1, \ldots, \mathbb{P}_K$ (line 1). It then initializes $d_s^P(v)$ and $d_t^P(v)$ for all $v \in V$ and $P \in \mathbb{P}$ (line 2).

(2) PDSBuild next computes the path degrees in a bottom-up manner and in parallel (lines 4–7). For each $P \in \mathbb{P}_i$, it applies HSplit/TSplit to obtain the sub-paths of P and, in parallel over each vertex $v \in V$, computes $d_s^P(v), d_t^P(v)$ using Eqs. (1)–(2).

(3) When $P \in \mathbb{P}$, the computed degrees are mapped to their ranks and used to update the corresponding rank buckets in the PathDS summaries defined in Definition 3.1 (line 8). It the end, PDSBuild returns $\{(\mathcal{S}_s^P, \mathcal{S}_t^P)\}_{P\in\mathbb{P}}$ (line 9).

Analysis. ⇒: analyze the time and space complexity of PDSBuild

6.2 Incremental PathDS Maintenance

Algorithm 2: FastCompress: Constructing CompressedDS

Input: FlatDS $f_P = [x_1, x_2, \ldots, x_m]$, bucket base b, prefix length n
Output: CompressedDS H and bucket maxima $\{M_k\}$

```
1 foreach x ∈ f_P do
2     k ← ⌈log_b x⌉;
3     π ← top-n most significant bits of x;
4     Insert (π, k) into H;
5     M_k ← max(M_k, x);
6 return H and {M_k}
```

a path pattern P, FlatDS records the exact number of matches of P originating from each valid start vertex. Conceptually, FlatDS corresponds to the most fine-grained statistic that PathDS can maintain and serves as the reference point for subsequent compression.

Collection Strategy. [To be filled: description of how FlatDS is collected efficiently for each path pattern P, including traversal strategy ...etc].

6.3 CompressedDS: Design and Representation

While FlatDS provides exact path statistics, it requires storing one count per start vertex. For large graphs or multiple path patterns, this results in substantial space overhead, making it impractical as a maintained summary.

A natural compression strategy is to construct a compact summary of FlatDS based on its ordered or aggregated distribution. Such approaches derive their representation from the globally arranged sequence of counts and often rely on cumulative relationships among values.

While this can produce concise summaries, the resulting structure exhibits cumulative dependencies: the representation of one position is influenced by other values in the ordered sequence. Consequently, a local modification may affect more than the modified count itself, potentially requiring adjustments beyond the changed value.

CompressedDS adopts a different design principle. Instead of aggregating multiple entries into a shared statistic, it compresses each FlatDS entry independently.

Compression Procedure. CompressedDS is constructed by applying entry-wise upper-bound compression to FlatDS. For each exact count x, the compression produces a compact descriptor (π, k), where $k = \lceil \log_b x \rceil$ denotes the logarithmic bucket index under base b, and π stores the top-n most significant bits of x. This representation preserves the order of magnitude of each entry while reducing storage overhead.

For each bucket k, we also record the maximum exact value M_k among all path counts assigned to that bucket. During cardinality estimation, all values mapped to bucket k are conservatively approximated by M_k.

Algorithm 2 presents the construction procedure. Because the algorithm scans FlatDS once and processes each entry independently, FastCompress runs in linear time in $|f_P|$ and supports streaming and parallel execution. Its cost is insensitive to the global distribution of path counts, and no cross-entry aggregation is required.

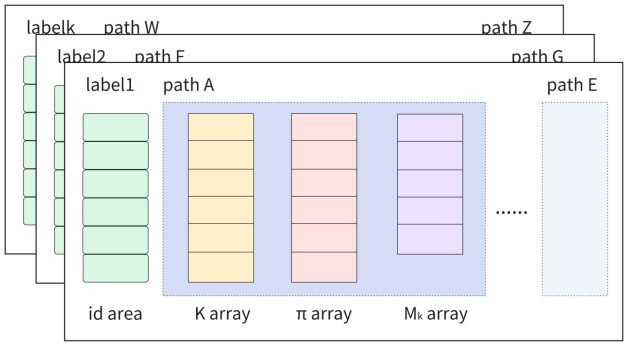

Figure 2: In-memory representation of CompressedDS.

Memory Layout. [To be filled: CompressedDS is stored as a contiguous array of fixed-size entries representing (π, k) pairs. Bucket-level maxima $\{M_k\}$ are stored in a separate compact array, indexed by bucket identifier. This separation allows efficient access to both per-entry information and bucket metadata while maintaining cache-friendly memory access patterns.]

6.4 Incremental Updates

With CompressedDS, path count changes can be handled incrementally. Since compression is independent across path counts, an update affects only the associated descriptor and its bucket metadata, without requiring global reconstruction.

When the exact count of a path instance changes by a small value Δ, the corresponding compressed entry (π, k) is updated as follows. First, we recover an estimate of the original value using the current bucket maximum M_k. Specifically, the value is restored as $x = M_k$. After applying the update, the new value $x' = x + \Delta$ is mapped to a possibly new bucket $k' = \lceil \log_b x' \rceil$, and a new prefix π' is extracted accordingly.

If $k' = k$, the update does not change the order of magnitude of the value, and no bucket migration is required. Only the bucket maximum M_k is updated if x' exceeds the current maximum. If $k' \neq k$, the entry migrates to the new bucket k', and both M_k and $M_{k'}$ are updated accordingly.

This update strategy ensures that each modification affects only constant-sized metadata, resulting in amortized constant update cost while preserving the upper-bound guarantee.

6.5 Upper-Bound Correctness and Estimation Accuracy

Upper-Bound Preservation. [proof]

Estimation Accuracy. [proof]

7 HANDLING CYCLES

8 EVALUATION

Experimental settings. We start with experimental settings.

Datasets and Queries.

Environment.

Exp-1: Estimation Accuracy.

Exp-2: Estimation Latency.

Exp-3: PDS Construction Latency.

Exp-4: PDS Maintenance Efficiency.

Exp-5: Scalability of DS Construction.

Exp-6: Micro Benchmarks.

Exp-6: Impact On End-to-End Performance.

(a) Scalability (vary thread count) **(b)** Scalability (vary scale factor) **(c)** End-to-end evaluation (LDBC) **(d)** End-to-end evaluation (IMDB)

(e) Scalability (vary thread count) **(f)** Scalability (vary scale factor) **(g)** End-to-end evaluation (LDBC) **(h)** End-to-end evaluation (IMDB)

(i) Scalability (vary thread count) **(j)** Scalability (vary scale factor) **(k)** End-to-end evaluation (LDBC) **(l)** End-to-end evaluation (IMDB)

(m) Scalability (vary thread count) **(n)** Scalability (vary scale factor) **(o)** End-to-end evaluation (LDBC) **(p)** End-to-end evaluation (IMDB)

Figure 3: Performance Evaluation

9 CONCLUSION

REFERENCES

[1] Walter Cai, Magdalena Balazinska, and Dan Suciu. 2019. Pessimistic Cardinality Estimation: Tighter Upper Bounds for Intermediate Join Cardinalities. In *Proceedings of the 2019 International Conference on Management of Data* (Amsterdam, Netherlands) *(SIGMOD '19)*. Association for Computing Machinery, New York, NY, USA, 18–35.

[2] Kyle B. Deeds, Dan Suciu, and Magdalena Balazinska. 2023. SafeBound: A Practical System for Generating Cardinality Bounds. *Proc. ACM Manag. Data* 1, 1, Article 53 (May 2023), 26 pages.

[3] Longbin Lai, Yufan Yang, Zhibin Wang, Yuxuan Liu, Haotian Ma, Sijie Shen, Bingqing Lyu, Xiaoli Zhou, Wenyuan Yu, Zhengping Qian, Chen Tian, Sheng Zhong, Yeh-Ching Chung, and Jingren Zhou. 2023. GLogS: Interactive Graph Pattern Matching Query At Large Scale. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. USENIX Association, Boston, MA, 53–69.

[4] Zhengdong Wang, Qiang Yin, and Longbin Lai. 2025. Path-Centric Cardinality Estimation for Subgraph Matching. *Proc. VLDB Endow.* 18, 9 (May 2025), 3063–3076.